STEM BUILDERS

Flow Controls

Accepting user input

Almost all interesting programs accept input from the user at some point. You can start accepting user input in your programs by using the `input()` function. The input function displays a messaget to the user describing the kind of input you are looking for, and then it waits for the user to enter a value. When the user presses Enter, the value is passed to your variable.

General syntax

The general case for accepting input looks something like this:

```
# Get some input from the user.
variable = input('Please enter a value: ')
# Do something with the value that was entered.
```

You need a variable that will hold whatever value the user enters, and you need a message that will be displayed to the user.

Example

In the following example, we have a list of names. We ask the user for a name, and we add it to our list of names.

```
# Start with a list containing several names.
names = ['guido', 'tim', 'jesse']

# Ask the user for a name.
new_name = input("Please tell me someone I should know: ")

# Add the new name to our list.
names.append(new_name)

# Show that the name has been added to the list.
print(names)
```

Output:

```
Please tell me someone I should know: jessica
['guido', 'tim', 'jesse', 'jessica']
```

Accepting input in Python 2.7

In Python 3, you always use `input()`. In Python 2.7, you need to use `raw_input()`:

```
# The same program, in Python 2.7

# Start with a list containing several names.
names = ['guido', 'tim', 'jesse']

# Ask the user for a name.
new_name = raw_input("Please tell me someone I should know: ")

# Add the new name to our list.
names.append(new_name)

# Show that the name has been added to the list.
print(names)
```

Output:

```
Please tell me someone I should know: jessica
['guido', 'tim', 'jesse', 'jessica']
```

The function `input()` will work in Python 2.7, but it's not good practice to use it. When you use the `input()` function in Python 2.7, Python runs the code that's entered. This is fine in controlled situations, but it's not a very safe practice overall.
If you're using Python 3, you have to use `input()`. If you're using Python 2.7, use `raw_input()`.

Exercises

Calculator

- Allow users to select an operation (addition, subtraction, multiplication or division)
- Allow them to input their desired numbers
- Perform the selected operation on said numbers
- Output the result

6. Flow Controls - While Loops and Input

While loops are really useful because they let your program run until a user decides to quit the program. They set up an infinite loop that runs until the user does something to end the loop. This section also introduces the first way to get input from your program's users.

What is a while loop?

A while loop tests an initial condition. If that condition is true, the loop starts executing. Every time the loop finishes, the condition is reevaluated. As long as the condition remains true, the loop keeps executing. As soon as the condition becomes false, the loop stops executing.

General syntax

```
# Set an initial condition.
game_active = True
# Set up the while loop.
while game_active:
    # Run the game.
    # At some point, the game ends and game_active will be set to False.
    #   When that happens, the loop will stop executing.
# Do anything else you want done after the loop runs.
```

- Every while loop needs an initial condition that starts out true.
- The `while` statement includes a condition to test.
- All of the code in the loop will run as long as the condition remains true.
- As soon as something in the loop changes the condition such that the test no longer passes, the loop stops executing.
- Any code that is defined after the loop will run at this point.

Example

Here is a simple example, showing how a game will stay active as long as the player has enough power.

```
# The player's power starts out at 5.
power = 5
# The player is allowed to keep playing as long as their power is over 0.
while power > 0:
     print("You are still playing, because your power is %d." % power)
     # Your game code would go here, which includes challenges that make it
     #   possible to lose power.
     # We can represent that by just taking away from the power.
     power = power - 1
print("\nOh no, your power dropped to 0! Game Over.")
```

Output:

```
You are still playing, because your power is 5.
You are still playing, because your power is 4.
You are still playing, because your power is 3.
You are still playing, because your power is 2.
You are still playing, because your power is 1.
Oh no, your power dropped to 0! Game Over.
```

Using while loops to keep your programs running

Most of the programs we use every day run until we tell them to quit, and in the background this is often done with a while loop. Here is an example of how to let the user enter an arbitrary number of names.

```
# Start with an empty list. You can 'seed' the list with

#  some predefined values if you like.
names = []

# Set new_name to something other than 'quit'.
new_name = ''

# Start a loop that will run until the user enters 'quit'.
while new_name != 'quit':
     # Ask the user for a name.
     new_name = input("Please tell me someone I should know, or enter 'quit': ")

     # Add the new name to our list.
     names.append(new_name)

# Show that the name has been added to the list.
print(names)
```

Output:

```
Please tell me someone I should know, or enter 'quit': guido
Please tell me someone I should know, or enter 'quit': jesse
Please tell me someone I should know, or enter 'quit': jessica
Please tell me someone I should know, or enter 'quit': tim
Please tell me someone I should know, or enter 'quit': quit
['guido', 'jesse', 'jessica', 'tim', 'quit']
```

That worked, except we ended up with the name 'quit' in our list. We can use a simple `if` test to eliminate this bug:

```
# Start with an empty list. You can 'seed' the list with

#  some predefined values if you like.
names = []

# Set new_name to something other than 'quit'.
new_name = ''

# Start a loop that will run until the user enters 'quit'.
while new_name != 'quit':
     # Ask the user for a name.
     new_name = input("Please tell me someone I should know, or enter 'quit': ")

     # Add the new name to our list.
     if new_name != 'quit':
        names.append(new_name)

# Show that the name has been added to the list.
print(names)
```

```
Output:
Please tell me someone I should know, or enter 'quit': guido
Please tell me someone I should know, or enter 'quit': jesse
Please tell me someone I should know, or enter 'quit': jessica
Please tell me someone I should know, or enter 'quit': tim
Please tell me someone I should know, or enter 'quit': quit
['guido', 'jesse', 'jessica', 'tim']
```

This is pretty cool! We now have a way to accept input from users while our programs run, and we have a way to let our programs run until our users are finished working.

Exercises

Password Validation

Write a Python program to check the validity of a password (input from users):
Validation

- Minimum length 6 characters
- Maximum length 12 charchers

Using while loops to make menus

You now have enough Python under your belt to offer users a set of choices, and then respond to those choices until they choose to quit. Let's look at a simple example, and then analyze the code:

```
# Give the user some context.
print("\nWelcome to the nature center. What would you like to do?")

# Set an initial value for choice other than the value for 'quit'.
choice = ''

# Start a loop that runs until the user enters the value for 'quit'.
while choice != 'q':
    # Give all the choices in a series of print statements.
    print("\n[1] Enter 1 to take a bicycle ride.")
    print("[2] Enter 2 to go for a run.")
    print("[3] Enter 3 to climb a mountain.")
    print("[q] Enter q to quit.")

    # Ask for the user's choice.
    choice = input("\nWhat would you like to do? ")

    # Respond to the user's choice.
    if choice == '1':
        print("\nHere's a bicycle. Have fun!\n")
    elif choice == '2':
        print("\nHere are some running shoes. Run fast!\n")
    elif choice == '3':
        print("\nHere's a map. Can you leave a trip plan for us?\n")
    elif choice == 'q':
        print("\nThanks for playing. See you later.\n")
    else:
        print("\nI don't understand that choice, please try again.\n")

# Print a message that we are all finished.
print("Thanks again, bye now.")
```

Output:

```
Welcome to the nature center. What would you like to do?

[1] Enter 1 to take a bicycle ride.
[2] Enter 2 to go for a run.
[3] Enter 3 to climb a mountain.
[q] Enter q to quit.
```

```
What would you like to do? 1

Here's a bicycle. Have fun!

[1] Enter 1 to take a bicycle ride.
[2] Enter 2 to go for a run.
[3] Enter 3 to climb a mountain.
[q] Enter q to quit.

What would you like to do? 3

Here's a map. Can you leave a trip plan for us?

[1] Enter 1 to take a bicycle ride.
[2] Enter 2 to go for a run.
[3] Enter 3 to climb a mountain.
[q] Enter q to quit.

What would you like to do? q

Thanks for playing. See you later.  Thanks again, bye now.
```

Our programs are getting rich enough now, that we could do many different things with them. Let's clean this up in one really useful way. There are three main choices here, so let's define a function for each of those items. This way, our menu code remains really simple even as we add more complicated code to the actions of riding a bicycle, going for a run, or climbing a mountain.

```
# Define the actions for each choice we want to offer.
def ride_bicycle():
    print("\nHere's a bicycle. Have fun!\n")

def go_running():
    print("\nHere are some running shoes. Run fast!\n")

def climb_mountain():
    print("\nHere's a map. Can you leave a trip plan for us?\n")

 # Give the user some context.
print("\nWelcome to the nature center. What would you like to do?")

# Set an initial value for choice other than the value for 'quit'.
choice = ''
# Start a loop that runs until the user enters the value for 'quit'.
while choice != 'q':
     # Give all the choices in a series of print statements.
```

```
    print("\n[1] Enter 1 to take a bicycle ride.")
    print("[2] Enter 2 to go for a run.")
    print("[3] Enter 3 to climb a mountain.")
    print("[q] Enter q to quit.")

    # Ask for the user's choice.
    choice = input("\nWhat would you like to do? ")

  # Respond to the user's choice.
  if choice == '1':
      ride_bicycle()
  elif choice == '2':
      go_running()
  elif choice == '3':
      climb_mountain()
  elif choice == 'q':
      print("\nThanks for playing. See you later.\n")
  else:
      print("\nI don't understand that choice, please try again.\n")

# Print a message that we are all finished.
print("Thanks again, bye now.")
```

Output:

```
Welcome to the nature center.
What would you like to do?

[1] Enter 1 to take a bicycle ride.
[2] Enter 2 to go for a run.
[3] Enter 3 to climb a mountain.
[q] Enter q to quit.

What would you like to do? 1
Here's a bicycle. Have fun!

[1] Enter 1 to take a bicycle ride.
[2] Enter 2 to go for a run.
[3] Enter 3 to climb a mountain.
[q] Enter q to quit.

What would you like to do? 3

Here's a map. Can you leave a trip plan for us?
```

```
[1] Enter 1 to take a bicycle ride.
[2] Enter 2 to go for a run.
[3] Enter 3 to climb a mountain.
[q] Enter q to quit.

What would you like to do? q

Thanks for playing. See you later.  Thanks again, bye now.
```

This is much cleaner code, and it gives us space to separate the details of taking an action from the act of choosing that action.

Using while loops to process items in a list

In the section on Lists, you saw that we can `pop()` items from a list. You can use a while list to pop items one at a time from one list, and work with them in whatever way you need. Let's look at an example where we process a list of unconfirmed users.

```
# Start with a list of unconfirmed users, and an empty list of confirmed users.
unconfirmed_users = ['ada', 'billy', 'clarence', 'daria']
confirmed_users = []
# Work through the list, and confirm each user.
while len(unconfirmed_users) > 0:
          # Get the latest unconfirmed user, and process them.
          current_user = unconfirmed_users.pop()
          print("Confirming user %s...confirmed!" % current_user.title())

          # Move the current user to the list of confirmed users.
          confirmed_users.append(current_user)

# Prove that we have finished confirming all users.
print("\nUnconfirmed users:")
for user in unconfirmed_users:
     print('- ' + user.title())

print("\nConfirmed users:")
for user in confirmed_users:
     print('- ' + user.title())
```

Output:

```
Confirming user Daria...confirmed!
Confirming user Clarence...confirmed!
Confirming user Billy...confirmed!
Confirming user Ada...confirmed!
```

```
Unconfirmed users:

Confirmed users:
- Daria
- Clarence
- Billy
- Ada
```

This works, but let's make one small improvement. The current program always works with the most recently added user. If users are joining faster than we can confirm them, we will leave some users behind. If we want to work on a 'first come, first served' model, or a 'first in first out' model, we can pop the first item in the list each time.

```
# Start with a list of unconfirmed users, and an empty list of confirmed users.
unconfirmed_users = ['ada', 'billy', 'clarence', 'daria']
confirmed_users = []

# Work through the list, and confirm each user.
while len(unconfirmed_users) > 0:
    # Get the latest unconfirmed user, and process them.
    current_user = unconfirmed_users.pop(0)
    print("Confirming user %s...confirmed!" % current_user.title())

    # Move the current user to the list of confirmed users.
    confirmed_users.append(current_user)

# Prove that we have finished confirming all users.
print("\nUnconfirmed users:")
for user in unconfirmed_users:
     print('- ' + user.title())

print("\nConfirmed users:")
for user in confirmed_users:
     print('- ' + user.title())
```

Output:

```
Confirming user Ada...confirmed!
Confirming user Billy...confirmed!
Confirming user Clarence...confirmed!
Confirming user Daria...confirmed!

Unconfirmed users:

Confirmed users:
```

```
- Ada
- Billy
- Clarence
- Daria
```

This is a little nicer, because we are sure to get to everyone, even when our program is running under a heavy load. We also preserve the order of people as they join our project. Notice that this all came about by adding *one character* to our program!

Accidental Infinite loops

Sometimes we want a while loop to run until a defined action is completed, such as emptying out a list. Sometimes we want a loop to run for an unknown period of time, for example when we are allowing users to give as much input as they want. What we rarely want, however, is a true 'runaway' infinite loop.

Take a look at the following example. Can you pick out why this loop will never stop?

```
current_number = 1
# Count up to 5, printing the number each time.
while current_number <= 5:
     print(current_number)
```

Output:

```
1 1 1 1 1 ...
```

You can try to run it on your computer, as long as you know how to interrupt runaway processes:

- On most systems, Ctrl-C will interrupt the currently running program.
- Use your pointer to close the terminal window.

The loop runs forever, because there is no way for the test condition to ever fail. The programmer probably meant to add a line that increments current_number by 1 each time through the loop:

```
current_number = 1
# Count up to 5, printing the number each time.
while current_number <= 5:
    print(current_number)
    current_number = current_number + 1
```

Output:

```
1 2 3 4 5
```

You will certainly make some loops run infinitely at some point. When you do, just interrupt the loop and figure out the logical error you made.

Infinite loops will not be a real problem until you have users who run your programs on their machines. You won't want infinite loops then, because your users would have to shut down your program, and they would consider it buggy and unreliable. Learn to spot infinite loops, and make sure they don't pop up in your polished programs later on.

Here is one more example of an accidental infinite loop:

```
current_number = 1
# Count up to 5, printing the number each time.
while current_number <= 5:
    print(current_number)
    current_number = current_number - 1
```

```
Output:
1 0 -1 -2 -3 ...
```

In this example, we accidentally started counting down. The value of `current_number` will always be less than 5, so the loop will run forever.

Overall Challenges

Fizzbuzz Challenge:

Fizzbuzz is a common job interview question in the programming world. The objective is as follows: for every number up to 100, output fizz if it is divisible by 3, buzz if divisible by 5, fizzbuzz if divisible by both, and nothing if divisible by either.

Your Program Should:
- Solve fizzbuzz
- Be under 10 lines of code

You should use:
- A loop
- Modulus (% in python)

Times Table in Python

1x1 = 1, 2x1 = 2, 10x10=100, you have all seen a times table before. Now you just have to make one.

Your Program Should:
- Output the times table up to 10x10
- Use loops
- Be under 5 lines of code

You Should Use:
- loops

Rolling the Dice:

This is a classic "roll the dice" program.

We will be using the random module for this,since we want to randomize the numberswe get from the dice.

We set two variables (min and max) , lowest and highest number of the dice.

We then use a while loop, so that the user can roll the dice again.

The roll_again can be set to any value, but here it's set to "yes" or "y",
but you can also add other variations to it.

Motivate to Innovate...

STEM BUILDERS Learning Center

MATH | ROBOTICS | TECHNOLOGY | EVENTS | CAMPS

contactus@stembuilders.com

www.stembuilders.com